BAC Blanc

ÉPREUVE D'ENSEIGNEMENT DE SPÉCIALITÉ NUMÉRIQUE ET SCIENCES INFORMATIQUES

Vendredi 9 janvier 2026

Durée de l'épreuve : 3 heures 30

L'usage de la calculatrice n'est pas autorisée.

Le candidat traite les quatre exercices proposés.

Ce sujet comporte 8 pages numérotées de 1/8 à 8/8.

Exercice 1 (5 points)

Notion abordée : gestion des processus et programmation orientée objet

On rappelle qu'un processus est l'instance d'un programme en cours d'exécution. Il est identifié par un numéro unique appelé PID. L'ordonnanceur est la composante du système d'exploitation qui gère l'allocation du processeur entre les différents processus. Nous allons nous intéresser à l'algorithme d'ordonnancement du tourniquet dont le fonctionnement est résumé ci-dessous :

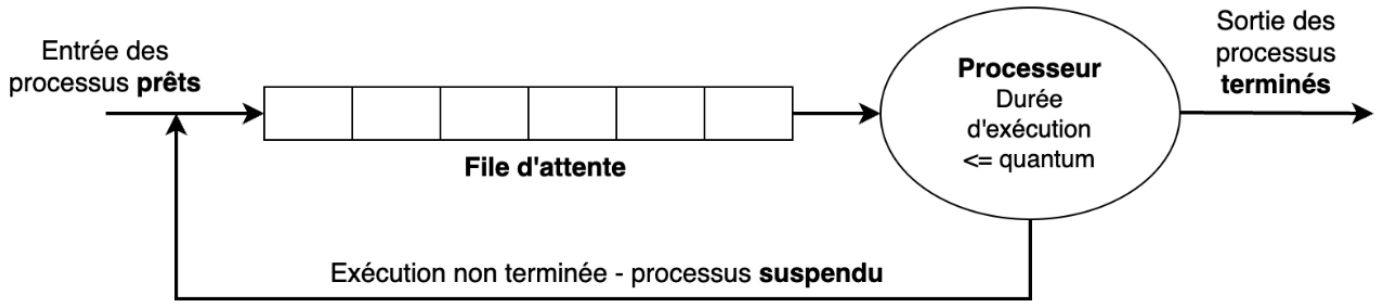

Schéma d'ordonnancement du tourniquet

- Les processus prêts à être exécutés sont placés dans une file d'attente selon leur ordre d'arrivée ;
- L'ordonnanceur alloue le processeur à chaque processus de la file d'attente un même nombre de cycles CPU, appelé **quantum** ;
- Si le processus n'est pas terminé au bout de ce temps, son exécution est suspendue et il est mis à la fin de la file d'attente ;
- Si le processus est terminé, il sort définitivement de la file d'attente.

1. On considère trois processus soumis à l'ordonnanceur au même instant pour lesquels on donne les informations ci-dessous :

PID	**Durée** (en cycles CPU)	**Ordre d'arrivée**
11	4	1
20	2	2
32	3	3

   (a) Si le quantum du tourniquet est d'un cycle CPU, recopier et compléter la suite des PID des processus dans l'ordre de leur exécution : 11, 20, 32, 11, ...

   correction :
   11, 20, 32, 11, 20, 32, 11, 32, 11

   (b) Donner la composition de la suite des PID lorsque le quantum du tourniquet est de deux cycles CPU.

   correction :
   Pour un quantum de 2 cycles CPU on obtient :
   11, 11, 20, 20, 32, 32, 11, 11, 32

2. L'objectif de la suite de l'exercice est d'implémenter en langage Python l'algorithme du tourniquet.
   Nous allons utiliser une liste pour simuler la file d'attente des processus et la classe `Processus` dont le constructeur est donné ci-dessous :

```
class Processus:
        def __init__(self, pid, duree):
                self.pid = pid
                self.duree = duree
                #Le nombre de cycle qui restent a faire
                self.reste_a_faire = duree
                self.etat = "Pret"
```

   Les états possibles d'un processus sont : « Prêt », « En cours d'exécution », « Suspendu» et « Terminé ».

   (a) Recopier et compléter l'instruction Python suivante permettant de créer la liste d'attente initiale des processus donnés dans le tableau précédent (le processus PID 11 est à l'indice 0 de la liste d'attente) :

   correction :

```
liste_attente = [Processus(11, 4), Processus(20, 2), Processus(32,3)]
```

   (b) Recopier (sans les commentaires) et compléter les trois méthodes suivantes de la classe `Processus` :

correction :

```
def execute_un_cycle(self):
        """ met a jour le reste a faire apres l'execution d'un cycle """
        self.reste_a_faire -= 1

def change_etat(self, nouvel_etat):
        """ change l'etat du processus avec la valeur passee en parametre
            """
        self.etat = nouvel_etat

def est_termine(self):
        """ Renvoie "True" si le processus est termine, "False" sinon, en
            se basant sur le reste a faire. """
        return self.reste_a_faire <= 0
```

(c) La fonction `tourniquet` ci-dessous implémente l'algorithme décrit dans l'exercice.
Elle prend en paramètre une liste d'objets `Processus` donnés par ordre d'arrivée et un nombre entier positif correspondant au quantum. La fonction renvoie la liste des PID dans l'ordre de leur exécution par le processeur.
Recopier et compléter sur la copie le code manquant.

Correction :

```
def tourniquet(liste_attente, quantum):
        ordre_execution = []
        while liste_attente != []:
                #on extrait le premier processus
                processus  = liste_attente.pop()
                processus.change_etat("En␣cours␣d'execution")
                compteur_tourniquet = 0
                while not processus.est_termine()  and compteur_tourniquet
                    < quantum:
                        ordre_execution.append(processus.pid)
                        processus.execute_un_cycle()
                        compteur_tourniquet = compteur_tourniquet + 1
                if not processus.est_termine() :
                        processus.change_etat("Suspendu")
                        liste_attente.append(processus)
                else:
                        processus.change_etat("Termine")
        return ordre_execution
```

Exercice 2 (4 points)

Cet exercice porte sur la programmation en Python et la manipulation des chaînes de caractères.

1. On définit la fonction `bonjour` par le code suivant :

```
def bonjour(nom)
  return 'Bonjour␣' + nom + '␣!'
```

Donner le résultat de l'exécution de `bonjour('Alan')`.

correction :

```
Bonjour Alan !
```

2. On exécute le programme suivant :

```
une_chaine='Bonjour'
x = (une_chaine[2] == une_chaine[3])
y = (une_chaine[4] == une_chaine[1])
```

Donner le type et les valeurs des variables `x` et `y`.

correction :
Les variables x et y sont des de type booléen.
x : False
y : True

3. Écrire une fonction `occurrences_lettre(une_chaine,une_lettre)` prenant en paramètres une chaîne `une_chaine` et une lettre `une_lettre` et renvoyant le nombre d'occurrences de `une_lettre` dans `une_chaine`.

correction :

```
def occurrences_lettre(une_chaine,une_lettre):
  nb_oc = 0
  for c in une_chaine:
    if c == une_lettre:
      nb_oc+=1
  return nb_oc
```

Exercice 3 (5 points)

Cet exercice porte sur la programmation orientée objet et les dictionnaires.

Dans le tableau ci-dessous, on donne les caractéristiques nutritionnelles, pour une quantité de 100 grammes, de quelques aliments.

	Lait entier UHT	Farine de blé	Huile de tournesol
Énergie (kcal)	65.1	343	900
Protéines (grammes)	3.32	11.7	0
Glucides (grammes)	4.85	69.3	0
Lipides (grammes)	3.63	0.8	100

Figure 1 : Caractéristiques nutritionnelles

Pour chaque aliment, on souhaite stocker les informations dans un objet de la classe `Aliment` définie ci-dessous, où `e`, `p`, `g` et `l` sont de type `float` et désignent respectivement les quantités d'énergie, de protéines, de glucides et de lipides de l'aliment.

```
class Aliment:
        def __init__(self, e, p, g, l):
                self.energie = e
                self.proteines = p
                self.glucides = g
                self.lipides = l
```

1. (a) Ecrire, à l'aide du tableau des caractéristiques nutritionnelles de la Figure 1, l'instruction en langage Python pour instancier l'objet lait.

   correction :

   ```
   lait = Aliment(65.1, 3.32, 4.85, 3.63)
   ```

   (b) Donner l'instruction qui permet d'obtenir la valeur 65.1 de l'objet lait instancié dans la question précédente.

   correction :

   ```
   lait.energie
   ```

   (c) Une erreur s'est introduite dans le tableau de la Figure 1 : la masse de protéines dans le lait est 3.4 au lieu de 3.32.
   Donner l'instruction qui modifie la masse de protéines de l'objet lait instancié dans la question 1.a.

   correction :

   ```
   lait.proteines = 3.4
   ```

2. On souhaite ajouter une méthode `energie_reelle` à la classe `Aliment` qui calcule l'énergie en kcal d'un aliment en fonction d'une masse donnée.

Par exemple :
Pour 245 grammes de lait, l'énergie réelle sera $245 \times 65.1 \div 100 = 159.495$ kcal. L'instruction `lait.energie_reelle(245)` renvoie alors 159.495.

Recopier la méthode ci-dessous et compléter les pointillés.

correction :

```
        def energie_reelle(self, masse):
                return masse*self.energie/100
```

3. On regroupe les caractéristiques nutritionnelles du tableau de la Figure 1 dans le dictionnaire suivant, les clés étant des chaînes de caractères donnant le nom de l'aliment et les valeurs associées des objets de la classe Aliment :

```
nutrition = {'lait' : Aliment(65.1,3.4,4.85,3.63),
             'farine' : Aliment(343,11.7,69.3,0.8),
             'huile' : Aliment(900,0,0,100)
            }
```

   (a) Donner l'instruction qui permet d'obtenir la valeur énergétique en kcal du lait à partir des données de ce dictionnaire.

   correction :

   `nutrition['lait'].energie`

   (b) Donner l'instruction qui permet d'obtenir la valeur énergétique réelle de 220 grammes de lait à partir des données de ce dictionnaire.

   correction :

   `nutrition['lait'].energie_reelle(220)`

4. Une recette de gâteau (sans oeuf) utilise les ingrédients suivants :
   - 230 g de farine ;
   - 220 g de lait ;
   - 100 g d'huile.

   Les quantités d'ingrédients, en grammes, sont regroupées dans le dictionnaire suivant :

```
recette_gateau={'lait' : 220, 'farine' :230, 'huile':100}
```

   Ecrire, en utilisant la classe `Aliment` et la méthode `energie_reelle`, les instructions nécessaires pour calculer l'énergie réelle totale du gâteau.

correction :

```
def calcul_energie_tot(recette, nutrition):
        """
        "recette" est un dictionnaire qui contient la recette
        "nutrition" est un dictionnaire qui contient des instances de la
        classe Aliment
        """
        energie_tot = 0
        for a,m in recette.items():
                energie_tot = energie_tot + nutrition[a].energie_reelle(m)
        return energie_tot
```

Exercice 4 (6 points)

Cet exercice porte sur les listes, les dictionnaires, les fonctions et la récursivité.

Nous souhaitons créer en langage Python un dictionnaire contenant un grand nombre de mots de façon à ce qu'une recherche dans ce dictionnaire soit la plus rapide possible.

Pour cela nous allons créer des groupes de mots, chaque groupe sera une liste Python associée à une clé unique dans le dictionnaire.

Voici un extrait du dictionnaire que nous souhaitons créer (les ... indiquent des éléments non listés dans cet extrait) :

```
d = {
    44 : ['ABAISSEMENT', 'ADMINISTRATEUR', ..., 'VERSETS'],
    74 : ['ABAISSER', 'ABLATION', ..., 'TROU'],
    243 : ['ABANDON', 'ALLEGRETTO', ..., 'ZIP'],
    36 : ['ABANDONNANT', 'ABOLITIONNISTE', ..., 'VOULAIT'],
    134 : ['ABANDONNE', 'AGNOSTICISME', ..., 'VOIES'],
    40 : ['ABANDONNENT', 'ACCOUCHEUSE', ..., 'YACK'],
    ...
}
```

Chaque clé sera un nombre entier positif et chaque valeur sera une liste de mots.

Partie A

Pour générer ces valeurs de clés, nous utiliserons une fonction dite de hachage, c'est- à-dire une fonction qui pour un mot donné calculera la clé qui lui sera associée. Nous choisissons une fonction de hachage simple qui consiste à additionner sur un octet les codes ASCII de chaque lettre de ce mot. Nous n'utiliserons que des lettres majuscules de l'alphabet, nous rappelons ici le code ASCII (en hexadécimal, c'est-à-dire en base 16) de chacune de ces lettres. Chaque valeur en hexadécimal est notée avec le préfixe `0x`, par exemple `0x15` correspond en décimal à $1 \times 16 + 5 \times 1 = 21$.

Table des codes `ASCII`					
'A' : `0x41`	'F' : `0x46`	'K' : `0x4B`	'P' : `0x50`	'U' : `0x55`	'Z' : `0x5A`
'B' : `0x42`	'G' : `0x47`	'L' : `0x4C`	'Q' : `0x51`	'V' : `0x56`	
'C' : `0x43`	'H' : `0x48`	'M' : `0x4D`	'R' : `0x52`	'W' : `0x57`	
'D' : `0x44`	'I' : `0x49`	'N' : `0x4E`	'S' : `0x53`	'X' : `0x58`	
'E' : `0x45`	'J' : `0x4A`	'O' : `0x4F`	'T' : `0x54`	'Y' : `0x59`	

Ainsi le mot 'NSI' aura pour clé `0xEA` puisque : `0x4E + 0x53 + 0x49 = 0xEA`, ou `234` en décimal.

Si la somme des codes ASCII ne tient pas sur un octet, seul l'octet de poids faible est conservé.

Par exemple : la somme des codes ASCII des lettres du mot 'ADMINISTRATEUR' est de `0x42C`, la clé qui lui sera associée sera donc `0x2C`, égale à `44` en décimal.

1. Trouver la valeur exprimée en hexadécimal de la clé qui sera associée au mot 'EW'.

   correction :
   E : `0x45 -> 69` en base décimale
   W : `0x57 -> 87` en base décimale
   0x45 + 0x57 = 0x9C

2. Comparer, sans les calculer, les valeurs des clés qui seront associées aux mots 'SAC' et 'CAS'. Justifier la réponse.

   correction :
   Les clés seront identiques car les 2 mots contiennent les mêmes lettre dans le désordre donc l somme des code acsii sera la même.

3. Voici le code de la fonction qui calcule cette clé pour un mot donné (nous rappelons que la fonction `ord` renvoie le code ASCII d'un caractère) :

   correction :

   ```
   1 def code_hachage(mot):
   2   somme = 0
   3   for caractere in mot:
   4     somme = somme + ord(caractere)
   5   return somme % 0x100
   ```

   Recopier et compléter les lignes 2, 3 et 4 de la fonction `code_hachage`.

4. Expliquer l'expression `somme % 0x100` dans le code ci-dessus et donner les valeurs possibles pour la clé.

correction :
% 0x100 permet de conserver uniquement l'octet de poids faible. La clé est donc comprise entre 0x0 et 0xFF (0 et 255 en décimal).

Partie B
Les listes de mots associées à chaque clé sont rangées dans l'ordre alphabétique, ce qui facilitera la recherche d'un mot par la suite.

Nous allons maintenant voir l'écriture d'une fonction permettant l'ajout d'un mot dans une liste de mots en maintenant un ordre alphabétique dans cette liste.

Nous rappelons qu'en langage Python la comparaison de chaînes de caractères est possible à l'aide des opérateurs classiques de comparaison : <, >, ==, >= et <= . Ces opérateurs utilisent l'ordre alphabétique.
Par exemple, l'expression booléenne `'ANNEE' < 'BATEAU'` vaut `True`.

Voici le code d'une fonction `ajouter_mot_liste(liste, mot)` qui a pour paramètres `liste`, une liste de chaînes de caractères (ce sont les mots) et `mot`, une chaîne de caractères correspondant au mot que l'on souhaite ajouter à liste. Cette fonction modifie `liste` en y ajoutant `mot` selon l'ordre alphabétique. Elle renvoie de plus `liste` après cet ajout.

```
1 def ajouter_mot_liste(liste, mot):
2   i = 0
3   while i < len(liste):
4     if mot < liste[i]:
5       # La methode "insert" permet d'ajouter un
6       # element a un indice donne dans "liste",
7       # les elements suivants sont decales.
8       liste.insert(i, mot)
9       return liste
10     i = i + 1
11   # La methode "append" permet d'ajouter un nouvel
12   # element a la fin de "liste".
13   liste.append(mot)
14   return liste
```

Voici des exemples d'utilisation de cette fonction :

```
>>> ajouter_mot_liste([],'NSI')
['NSI']
>>> ajouter_mot_liste(['NSI'],'PYTHON')
['NSI','PYTHON']
>>> ajouter_mot_liste(['NSI','PYTHON'],'OBJET')
['NSI','OBJET','PYTHON']
>>> ajouter_mot_liste(['NSI','OBJET','PYTHON'],'RAM')
['NSI','OBJET','PYTHON','RAM']
```

1. Déterminer, dans le pire cas, l'ordre de grandeur du nombre de comparaisons entre chaînes de caractères effectuées par un appel à `ajouter_mot_liste`.
   On exprimera le résultat en fonction de n, le nombre de mots présents dans `liste`.

   correction :
   On effectue n comparaisons (complexité O(n))

2. Nous souhaitons écrire une fonction qui permette l'ajout d'un mot dans le dictionnaire décrit au début de l'exercice (qui associe à chaque clé entière possible une liste de mots).
   Rappeler avec quelle expression Python on peut tester si une clé `c` est déjà présente dans un dictionnaire `dico`.

   correction :

   ```
   c in dico
   ```

3. Écrire un code pour la fonction `ajouter_mot_dict(dict_mots, mot)` qui a pour paramètres `dict_mots`, un dictionnaire qui associe à chaque clé entière possible une liste de mots, et `mot` le mot à ajouter dans `dict_mots`, et qui réalise cet ajout (mais ne renvoie aucune valeur). On utilisera les fonctions `code_hachage` et `ajouter_mot_liste`.

   correction :

   ```
   def ajouter_mot_dict(dict_mots, mot):
     cle = code_hachage(mot)
     if cle in dict_mots:
           dict_mots[cle] = ajouter_mot_liste(dict_mots[cle],mot)
     else:
           dict_mots[cle] = [mot]
   ```

Partie C

Nous allons maintenant voir comment faire la recherche d'un mot dans notre dictionnaire.

Nous souhaitons une recherche rapide et nous allons profiter du fait que les mots sont classés dans l'ordre alphabétique dans les listes du dictionnaire.

Nous prendrons une approche dichotomique pour écrire une fonction `est_present(liste, mot, debut, fin)` qui a pour paramètres :

- `liste`, une liste de mots classés dans l'ordre alphabétique ;
- `mot`, le mot à rechercher ;
- `debut` et `fin`, les indices entre lesquels la recherche se fait dans liste.

Cette fonction renverra `True` si `liste` contient `mot` entre les indices `debut` (inclus) et `fin` (exclus), et renverra `False` sinon.

Voici son code :

```
1 def est_present(liste, mot, debut, fin):
2   if debut > fin:
3     return False
4   milieu = (debut + fin) // 2
5   if liste[milieu] > mot:
6     return est_present(liste, mot, 0, milieu - 1)
7   elif liste[milieu] < mot:
8     return est_present(liste, mot, milieu + 1, fin)
9   else:
10    return True
```

Nous testons cette fonction ainsi :

```
>>> liste_mots = ['FONCTION', 'NSI', 'PYTHON', 'OBJET', 'RAM']
>>> est_present(liste_mots, 'NSI', 0, len(liste_mots))
True
```

1. Donner, lors de l'exécution de l'exemple précédent, les valeurs des paramètres `debut` et `fin` prises lors de chaque appel récursif (appel initial inclus) de la fonction `est_present`.

   correction :

2. Expliquer pourquoi la méthode dichotomique permet d'effectuer, en général, moins d'opérations qu'une recherche simple qui consisterait à comparer un par un les mots de la liste avec le mot cherché.

   correction :
   Appel initial : début = 0, fin = 5
   premier appel récursif : début = 0, fin = 1
   deuxième appel récursif : début = 1, fin = 1

3. Donner l'ordre de grandeur du nombre de comparaisons de mots effectuées par la méthode dichotomique sur une liste de longueur n ?

   correction :
   On a une complexité en log(n)

4. Écrire un code pour la fonction `mot_present(dict_mots, mot)` qui a pour paramètres `dict_mots` , un dictionnaire qui associe à chaque clé entière possible une liste de mots, et `mot`, le mot à rechercher dans `dict_mots`, et qui renvoie `True` si le mot est présent et `False` sinon. On utilisera les fonctions `code_hachage` et `est_present`.

   correction :

```
def mot_present(dict_mots, mot)
  cle = code_hachage(mot)
  if cle in dict_mots:
        return est_present(dict_mots[cle],mot,0,len(dict_mots[cle]))
  else:
        return False
```